

RAPPORT PYTHON

Informatique 3 (API2)

Année universitaire : 2025 – 2026

—

Rapport technique – Implémentation Python

Réalisé par :

AMBDOULFATTAH MOHAMED HALIM (N °20)

AKEBBAD MOUHSSIN (N°18)

AMDINI AYOUB (N°21)

Introduction :

Dans le cadre du module Programmation en Python, ce mini-projet a pour objectif la mise en œuvre d'algorithmes élémentaires de traitement d'images numériques. Une image est représentée sur machine sous forme matricielle, ce qui permet d'appliquer des transformations mathématiques et géométriques sur ses pixels.

Le travail réalisé consiste à développer un programme interactif en Python permettant de manipuler des images en noir et blanc, en niveaux de gris et en couleur RGB, conformément à l'énoncé fourni.

Organisation générale du programme

Le programme est structuré autour d'un ensemble de fonctions indépendantes, chacune correspondant à une opération précise du traitement d'images. Un menu principal permet à l'utilisateur de choisir l'opération souhaitée et d'exécuter le traitement associé.

➤ Fonction lectureImage:

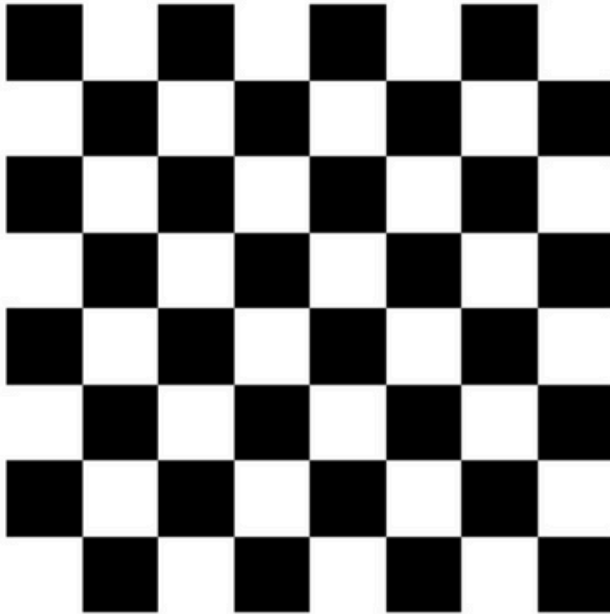
```
def lectureImage(Img):  
    Img = Image.open(Img).convert("L")  
    return np.array(Img)
```

Cette fonction permet de charger une image à partir de son chemin d'accès et de la convertir en niveaux de gris. L'image est ensuite transformée en une matrice NumPy représentant les intensités des pixels. Cette représentation matricielle facilite les opérations de traitement d'image telles que l'inversion, la symétrie ou le calcul de caractéristiques comme la luminance et le contraste.

➤ Fonction Image Noir Blanche:

```
# définir l'image comme damier
def image_noir_blanche(h, l):
    Img = np.ones((h, l))
    for i in range(h):
        for j in range(l):
            Img[i, j] = (i + j) % 2
    return Img
```

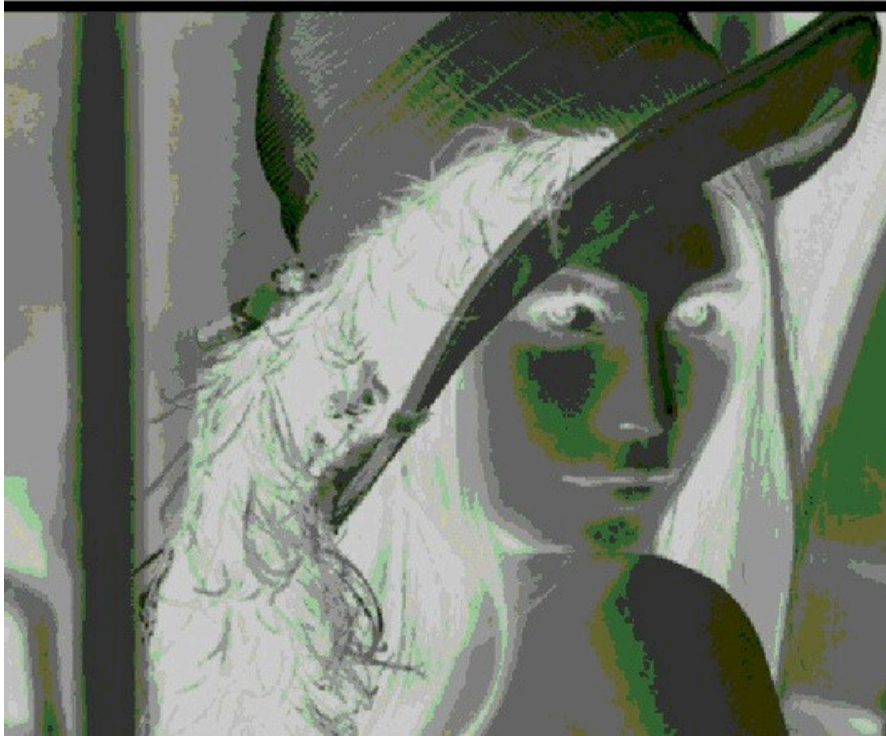
Image blanche_noir



Cette fonction génère une image noire et blanche de dimensions données. Les valeurs des pixels sont calculées de manière alternée afin de produire un motif en damier. L'image obtenue est représentée sous forme d'une matrice contenant uniquement les valeurs 0 et 1, ce qui permet de tester les différentes opérations de traitement sans utiliser d'images externes.

➤ Fonction inverser l'Image:

```
# inverser l'image
def negatif(Img):
    return 1 - Img
```



Cette fonction calcule le négatif d'une image binaire. Chaque pixel de l'image d'entrée est inversé afin d'obtenir une image où les zones noires deviennent blanches et inversement. L'image résultante conserve les mêmes dimensions que l'image originale.

➤ **Fonction de Luminance:**

```
# fonction de luminance
def luminance(Img):
    h = len(Img)
    l = len(Img[0])
    somme = 0
    for i in range(h):
        for j in range(l):
            somme += Img[i, j]
    moyenne = somme / (h * l)
    return moyenne
```

Cette fonction calcule la luminance moyenne d'une image en parcourant l'ensemble de ses pixels. Elle additionne les valeurs de tous les pixels et divise le résultat par le nombre total de pixels. La valeur obtenue représente le niveau moyen de luminosité de l'image.

➤ **Fonction d'affichage:**


```
# affichage
def afficher_image(Img):
    if len(Img.shape) == 3 and Img.shape[0] == 3:
        plt.imshow(np.transpose(Img, (1, 2, 0)))
    else:
        plt.imshow(Img, cmap='gray')
    plt.title("Image blanche_noir")
    plt.axis("off")
    plt.show()
```

Cette fonction permet d'afficher une image à l'écran en utilisant la bibliothèque Matplotlib. Elle distingue automatiquement entre une image en niveaux de gris et une image RGB en analysant la dimension de la matrice. Dans le cas d'une image RGB, les dimensions sont réorganisées afin de respecter le format attendu pour l'affichage. L'image est affichée sans axes afin de mettre en évidence uniquement le contenu visuel.

➤ Fonction contraste de l'image:

```
# Retourne le contraste d'une image
def constrast(Img):
    h = len(Img)
    l = len(Img[0])
    lum = luminance(Img)

    s = 0
    for i in range(h):
        for j in range(l):
            s += (Img[i, j] - lum) ** 2
    return s / (h * l)
```

Cette fonction calcule le contraste d'une image à partir de la luminance moyenne. Elle mesure la dispersion des valeurs des pixels autour de cette luminance en calculant la moyenne des carrés des écarts. Une valeur de contraste élevée indique une image avec de fortes variations de luminosité, tandis qu'une valeur faible correspond à une image plus uniforme.

➤ Fonction Profondeur:

```
# Retourne la valeur maximale d'un pixel dans l'image Img
def profondeur(Img):
    maxi = Img[0][0]
    for i in range(len(Img)):
        for j in range(len(Img[0])):
            if Img[i, j] > maxi:
                maxi = Img[i, j]
    return maxi
```

Cette fonction permet de déterminer la valeur maximale d'un pixel dans une image. Elle parcourt l'ensemble de la matrice représentant l'image et compare chaque pixel afin d'identifier la valeur la plus élevée. Le résultat correspond à l'intensité maximale présente dans l'image, ce qui donne une indication sur la plage de valeurs utilisée.

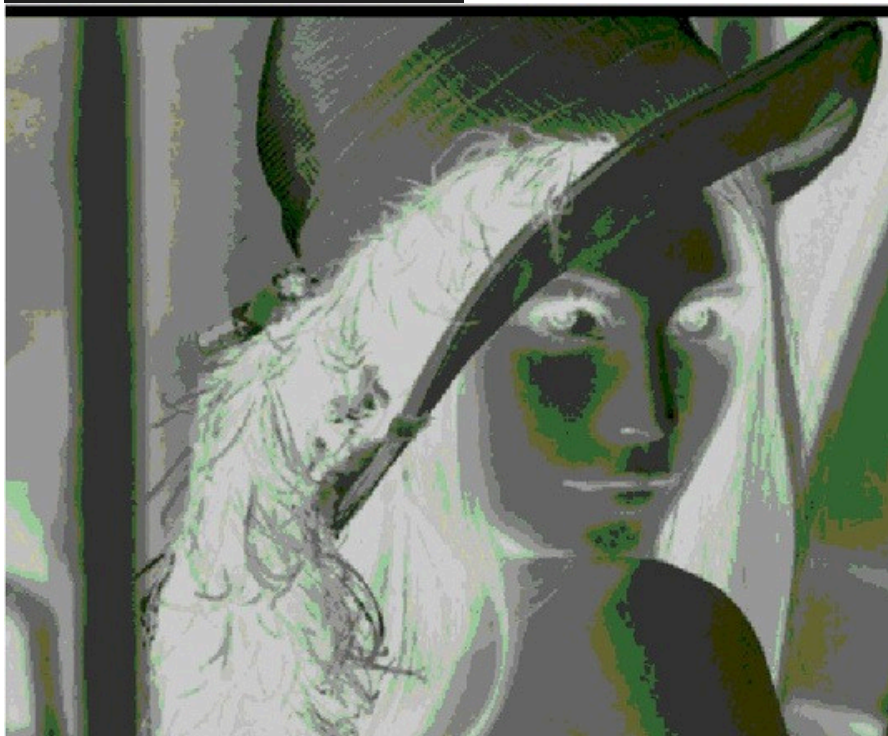
➤ Fonction ouverture d'une image:

```
def Ouvrir(A):  
    return lectureImage(A)
```

Cette fonction permet d'ouvrir une image à partir de son chemin d'accès en utilisant la fonction de chargement définie précédemment. Elle sert d'interface simple pour l'ouverture d'une image dans le programme principal, facilitant son utilisation dans le menu et dans les différentes opérations de traitement. L'image retournée est directement exploitée sous forme de matrice.

➤ Fonction D'Inversion d'une image en niveaux Gris:

```
def inverser(img):  
    p = 255  
    return p - img
```



Cette fonction réalise l'inversion d'une image en niveaux de gris codée sur 8 bits. Chaque pixel est transformé en soustrayant sa valeur à 255, ce qui produit l'image négative correspondante. Les zones claires deviennent sombres et les zones sombres deviennent claires, tout en conservant les dimensions de l'image originale.

➤ Fonction de Symetrie vertical :

```
# Symétrie verticale (gauche-droite)
def flipH(img):
    L = len(img)
    C = len(img[0])

    for i in range(L):
        for j in range(C // 2):
            t = img[i][j]
            img[i][j] = img[i][C - 1 - j]
            img[i][C - 1 - j] = t

    return img
```



Cette fonction réalise une symétrie verticale de l'image, c'est-à-dire un retournement gauche-droite par rapport à un axe vertical passant par le centre de l'image. Les pixels situés à gauche sont échangés avec ceux situés à droite sur chaque ligne. L'image obtenue possède les mêmes dimensions que l'image initiale, mais son contenu est inversé horizontalement.

➤ Fonction de poser verticalement deux images:

```
# Poser verticalement img1 au-dessus de img2
def poserv(img1, img2):
    L1 = len(img1)
    C1 = len(img1[0])
    L2 = len(img2)
    C2 = len(img2[0])

    if C1 != C2:
        return None

    resultat = np.zeros((L1 + L2, C1))

    for i in range(L1):
        for j in range(C1):
            resultat[i][j] = img1[i][j]

    for i in range(L2):
        for j in range(C2):
            resultat[i + L1][j] = img2[i][j]

    return resultat
```



Cette fonction permet d'assembler deux images verticalement en plaçant la première image au-dessus de la seconde. Les deux images doivent avoir le même nombre de colonnes afin que l'assemblage soit possible. Une nouvelle image est créée pour contenir les deux images concaténées, dont la hauteur correspond à la somme des hauteurs des images d'entrée.

➤ Fonction de poser horizontalement deux images:

```
# Poser horizontalement img2 à droite de img1
def poserH(img1, img2):
    L1 = len(img1)
    C1 = len(img1[0])
    L2 = len(img2)
    C2 = len(img2[0])

    if L1 != L2:
        return None

    resultat = np.zeros((L1, C1 + C2))

    for i in range(L1):
        for j in range(C1):
            resultat[i][j] = img1[i][j]

        for j in range(C2):
            resultat[i][j + C1] = img2[i][j]

    return resultat
```



Cette fonction réalise l'assemblage horizontal de deux images en plaçant la deuxième image à droite de la première. Les deux images doivent avoir le même nombre de lignes. Le résultat est une nouvelle image dont la largeur correspond à la somme des largeurs des deux images d'entrée.

➤ Fonction d'initialisation d'un image RGB:

```
def initImageRGB(n, m):  
    imageRGB = np.zeros((3, n, m), dtype=int)  
    for c in range(3):  
        for i in range(n):  
            for j in range(m):  
                imageRGB[c, i, j] = randrange(256)  
    return imageRGB
```



Cette fonction initialise une image RGB de dimensions données. L'image est représentée par une matrice tridimensionnelle composée de trois plans correspondant aux canaux Rouge, Vert et Bleu. Chaque pixel est initialisé avec une valeur aléatoire comprise entre 0 et 255, ce qui permet de générer une image couleur artificielle pour les tests et les manipulations.

➤ Fonction de Symetrie horizontale d'un image RGB:

```
# symétrie horizontale (haut-bas)
def symetrie_horizontale(img):
    return img[::-1, :]
```



Cette fonction réalise une symétrie horizontale de l'image, correspondant à un retournement haut-bas. Les lignes de l'image sont inversées afin d'obtenir une image miroir par rapport à un axe horizontal passant par le centre. L'image résultante conserve les mêmes dimensions que l'image initiale.

➤ Fonction de Symetrie verticale d'un image RGB:

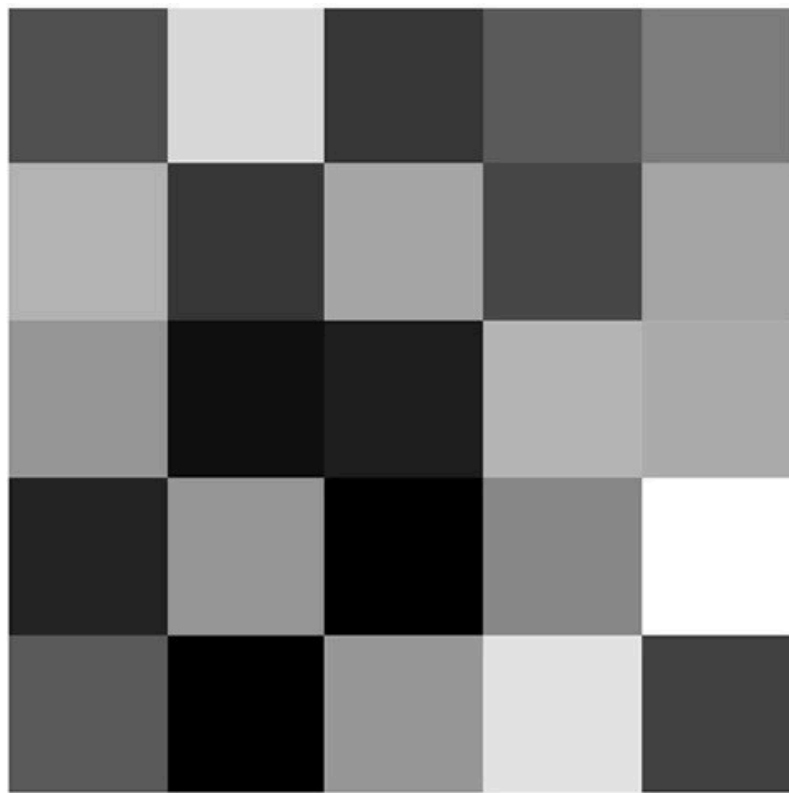
```
# symétrie verticale (gauche-droite)
def symetrie_verticale(img):
    return img[:, ::-1]
```



Cette fonction effectue une symétrie verticale de l'image en inversant l'ordre des colonnes. Elle produit un effet miroir gauche-droite à l'aide d'une opération de découpage sur la matrice NumPy. Le résultat est une image inversée horizontalement, sans modification de sa taille.

➤ Fonction image Gris:

```
def grayscale(imageRGB):  
    m, n = imageRGB.shape[1], imageRGB.shape[2]  
    gris = np.zeros((m, n), dtype=int)  
  
    for i in range(m):  
        for j in range(n):  
            R = imageRGB[0, i, j]  
            G = imageRGB[1, i, j]  
            B = imageRGB[2, i, j]  
            gris[i, j] = (max(R, G, B) + min(R, G, B)) // 2  
  
    return gris
```

Cette fonction transforme une image couleur RGB en une image en niveaux de gris. Pour chaque pixel, les composantes rouge, verte et bleue sont utilisées afin de calculer une valeur unique représentant l'intensité lumineuse. L'image obtenue est une matrice bidimensionnelle correspondant à l'image originale sans information de couleur.

➤ Fonction Menu:

```
# menu
def menu():
    print("1. Ouvrir une image")
    print("2. Créer une image noire et blanche")
    print("3. construit le négatif de l'image donnée")
    print("4. ouvrir l'image A")
    print("5. Inverser une image")
    print("6. Symétrie d'axe vertical passant par le milieu de l'image")
    print("7. Poser verticalement deux images")
    print("8. Poser horizontalement deux images")
    print("9. l'affichage des valeurs du matrice M de l'images RGB")
    print("10. Initialiser et renvoyer une image RGB")
    print("11. Symétrie horizontale d'une image RGB")
    print("11. Symétrie verticale d'une image RGB")
    print("12. Convertir une image RGB en niveaux de gris")
    print("13. Quitter")

    choix = int(input("Veuillez choisir une option: "))
    return choix
```

```

while True:
    choix = menu()
    if choix == 13:
        print("Au revoir!")
        break

    elif choix == 1:
        try:
            m = input("entrez le chemin d'accès de l'image en supprimant les guillemets ")
            m = lectureImage(m)
            afficher_image(m)
        except:
            print("image non trouvé, veuillez écrire un chemin d'accès valide ")

    elif choix == 2:
        h = int(input("Entrez la hauteur de l'image: "))
        l = int(input("Entrez la largeur de l'image: "))
        img = image_noir_blanche(h, l)
        afficher_image(img)
    elif choix == 3:
        chemin = input("entrez le chemin d'accès de l'image A en supprimant les guillemets: ")
        img = lectureImage(chemin)
        neg = negatif(img)
        afficher_image(neg)
    elif choix == 4:

```

Le programme repose sur un menu interactif qui permet à l'utilisateur d'exécuter différentes opérations de traitement d'images. Ce menu s'affiche dans la console et propose une liste numérotée d'actions. À chaque itération, l'utilisateur choisit une option en saisissant un numéro, ce qui déclenche l'exécution de la fonction correspondante.

Le menu est intégré dans une boucle `while True`, ce qui permet au programme de rester actif tant que l'utilisateur ne choisit pas l'option de sortie. Cette structure assure une utilisation continue et évite de relancer le programme après chaque opération.

Les premières options du menu sont dédiées à la **manipulation d'images en niveaux de gris**. L'utilisateur peut ouvrir une image à partir de son chemin d'accès, créer une image noire et blanche artificielle, calculer le négatif d'une image ou encore appliquer une inversion des niveaux de gris. Ces opérations permettent de travailler directement sur des matrices représentant les intensités lumineuses des pixels.

Le menu propose également des opérations de **transformation géométrique**, telles que la symétrie verticale et horizontale. Ces transformations modifient la disposition des pixels sans altérer les dimensions de l'image, ce qui permet d'observer l'effet des symétries sur le contenu visuel.

Certaines options sont consacrées à la **combinaison de plusieurs images**. Le programme permet de poser deux images verticalement ou horizontalement, sous réserve que leurs dimensions soient compatibles. Dans ces cas, une nouvelle image est créée afin de contenir le résultat de l'assemblage.

Une partie du menu est réservée aux **images couleur RGB**. L'utilisateur peut initialiser une image RGB avec des valeurs aléatoires, visualiser sa structure matricielle, appliquer des symétries et convertir cette image en niveaux de gris. Ces fonctionnalités permettent de comprendre la différence entre une image monochrome et une image couleur, ainsi que la manière dont les canaux RGB sont exploités.

Enfin, l'option de sortie permet d'interrompre l'exécution du programme de manière contrôlée.

Lorsque cette option est sélectionnée, la boucle principale s'arrête et un message de fin est affiché, indiquant la fermeture correcte de l'application.

Dans l'ensemble, le menu joue un rôle central dans l'organisation du programme. Il assure la liaison entre les différentes fonctions développées et permet de tester l'ensemble des traitements d'image de manière simple, interactive et structurée.